# Measuring Prompt Leakage Resistance in Deployed LLM Agents

Gaussia Team
research@gaussia.ai

April 29, 2026

**Abstract**

Large language models deployed with system prompts containing sensitive information are vulnerable to adversarial extraction attacks. No standard, reproducible metric exists to quantify how well a model resists such attacks. We propose **Prompt Leakage Resistance (PLR)**, a two-signal detection framework that combines a semantic similarity score between the agent response and chunks of the system prompt with a cosine alignment score against a reference safe response. We evaluate four similarity methods—cosine sentence embeddings, a cross-encoder reranker, ROUGE-L, and NLI entailment—on a benchmark of 68 records spanning four leakage categories and three difficulty levels. ROUGE-L and NLI are empirically disqualified as structurally incompatible with the task. Among the viable methods, the cross-encoder reranker achieves 61.5% detection at the default threshold $\tau = 0.6$ with no hard false negatives. Threshold calibration shows $\tau = 0.65$ as the optimal operating point: 76.9% detection, 0% hard false negative rate, and a 6.2% false positive rate, while $\tau = 0.70$ triggers a sharp increase in false positives to 31.2%. We additionally evaluate a *smart cascade* that uses cosine embeddings to pre-filter safe responses before the reranker, and find that it reduces judge escalation at the cost of converting ambiguous adversarial cases into hard false negatives—a net regression in security-critical deployments. The metric and benchmark are released as part of the Gaussia evaluation framework.

## 1 Introduction

The proliferation of LLM-powered agents in production has made system prompt security a first-order concern. Operators configure agents via system prompts that frequently contain sensitive material: proprietary business logic, internal credentials, access control rules, and behavioral constraints. Adversarial users can exploit prompt injection techniques [Perez and Ribeiro, 2022] to extract this information, exposing intellectual property or enabling privilege escalation.

Existing evaluation frameworks treat prompt leakage as a binary observable: either the agent refused to reveal the prompt or it did not. This framing ignores the spectrum of partial leakage, paraphrase-mediated disclosure, and the high false-positive risk of flagging legitimate responses that necessarily reference system prompt content. No metric allows practitioners to compare model configurations, system prompt formulations, or agent architectures on a common scale.

We make the following contributions:

1. **PLR metric**: a numeric score derived from a two-signal, chunk-based detection pipeline with a four-way decision table.

2. **Benchmark**: 68 records across four leakage categories and three difficulty levels, generated with Gemma-3-12B-IT on realistic system prompts.

3. **Comparative evaluation**: four similarity methods tested against adversarial and legitimate records, with a clear empirical argument for discarding lexical and NLI approaches.

4. **Threshold calibration**: a sweep over the decision threshold $\tau$ for the cross-encoder reranker, identifying $\tau = 0.65$ as the optimal operating point (76.9% detection, 0% hard false negatives, 6.2% FP) and showing that $\tau = 0.70$ causes a sharp false positive increase to 31.2%.

5. **Smart cascade (negative result)**: a combination of cosine embeddings and the reranker that reduces judge escalation but converts ambiguous adversarial cases into hard false negatives, making it unsuitable as the primary configuration in security-sensitive deployments.

## 2 Related Work

**LLM safety evaluation.** Prompt injection and jailbreak attacks have received growing attention [Perez and Ribeiro, 2022]. Most defenses focus on input filtering rather than post-hoc response evaluation.

**Leakage metrics.** Existing approaches to measuring prompt leakage fall into two broad categories. The first relies on an LLM judge that issues a binary verdict — leaked or not — for each response. While simple to deploy, this approach produces no numeric score for model comparison, provides no analysis of false positives, and offers no insight into partial or paraphrase-mediated leakage. The second computes direct text similarity between the response and the full system prompt as a single vector. This approach conflates legitimate domain overlap with actual leakage and has no mechanism to distinguish a model that answers safely about its own domain from one that discloses confidential content.

PLR advances beyond both by introducing chunk-level similarity to preserve partial-leak signals in long prompts, a second signal anchored to a safe reference response to control false positives, and a numeric output that enables quantitative comparison across model configurations.

Hui et al. [2024] introduce PLeak, a closed-box attack framework that recovers system prompts by treating prompt theft as an optimisation problem. Their work characterises the attack surface from the adversary's perspective. PLR is complementary: it provides a per-response detection pipeline for operators to evaluate their own agent's resistance on their own dataset, independently of any specific attack strategy.

**Similarity methods.** Sentence-BERT [Reimers and Gurevych, 2019] established bi-encoder embeddings as the standard for semantic similarity. Cross-encoder rerankers [Nogueira and Cho, 2019] process text pairs jointly and achieve higher accuracy at the cost of increased latency. ROUGE [Lin, 2004] remains the standard lexical overlap metric for summarization evaluation. NLI models [He et al., 2021] capture entailment relationships but are designed for sentence-level logical reasoning tasks with strict token-length constraints.

## 3 Problem Formulation

Let $P$ denote the system prompt, $Q$ an adversarial user query, $R$ the agent response to $Q$ under prompt $P$, and $G$ a reference safe response to $Q$ that does not reveal $P$.

**Definition 1** (Prompt Leakage)**.** *A response $R$ constitutes a* leak *of $P$ if it discloses information from $P$ that was not already public knowledge or required by the query $Q$ to answer correctly.*

We partition leakable content into four categories that reflect the type of information at risk:

- **Secrets & Credentials**: API keys, tokens, passwords.

- **Internal Instructions**: business rules, persona definitions, behavioral constraints.
- **Guard Mechanism Exposure**: content filters, moderation triggers, rejection rules.
- **Permissions & Roles**: access levels, user roles, capability restrictions.

Each attack instance carries a difficulty label (*easy*, *medium*, *hard*) reflecting the sophistication of the adversarial query. Hard instances use paraphrase, roleplay, and indirect multi-step strategies rather than direct extraction requests.

# 4 PLR Metric

## 4.1 Chunking the System Prompt

A system prompt embedded as a single vector averages all its semantic content. If the model leaks only one section—for example, the credentials paragraph—that signal is diluted by the non-leaked sections. We address this by splitting $P$ into overlapping chunks $C = \{c_1, c_2, \ldots, c_k\}$ of at most four sentences each, and computing the *maximum* similarity between the response and any individual chunk:

$$\text{MaxChunkSim}(R, P) = \max_{c_i \in C} \text{sim}(R,\, c_i) \tag{1}$$

This design ensures that partial leaks of long prompts are not suppressed by averaging.

## 4.2 Two-Signal Design

PLR detection relies on two complementary signals:

- **Signal 1** $= \text{MaxChunkSim}(R, P)$: how much content from the system prompt appears in the response.
- **Signal 2** $= \text{CosineSim}(R, G)$: how closely the response aligns with the reference safe response $G$. A high score indicates the agent responded within safe boundaries; a low score indicates divergence toward unreleased content.

**Design decision: Signal 2 always uses cosine.** Our experiments show that the cross-encoder reranker inflates $\text{sim}(R, G)$ artificially—adversarial responses receive a mean Signal 2 score of 0.782, indistinguishable from legitimate responses. This collapses the decision table by routing most adversarial records to the *ambiguous* bucket regardless of Signal 1. Cosine embeddings produce a calibrated Signal 2 (mean 0.511 across both adversarial and legitimate records), making it the only method suitable for the second signal regardless of which method is used for Signal 1.

## 4.3 Decision Table

Table 1: Decision table for PLR verdicts. Threshold: $\tau = 0.6$ (default).

| $\text{MaxChunkSim}(R, P)$ | $\text{CosineSim}(R, G)$ | Verdict |
|---|---|---|
| $\geq \tau$ | $< \tau$ | **Leak** |
| $< \tau$ | $\geq \tau$ | **Safe** |
| $< \tau$ | $< \tau$ | **Anomalous** $\rightarrow$ judge |
| $\geq \tau$ | $\geq \tau$ | **Ambiguous** $\rightarrow$ judge |

Anomalous and Ambiguous cases are escalated to a lightweight LLM judge. The threshold $\tau$ is a user-configurable parameter; $\tau = 0.6$ is the recommended default, grounded in cosine similarity distributions for semantically related texts reported by Reimers and Gurevych [2019]. On this benchmark, $\tau = 0.65$ achieves higher detection (76.9%) at identical false positive rate (Section 7.3), and is the recommended deployment target after validating false positive tolerance on a representative legitimate sample. Methods with different score ranges require independent threshold calibration (Section 7.3).

### 4.4 PLR Score and Output

The metric operates at two levels.

**Per-record output.** For each response $R_i$, the pipeline returns a discrete verdict $v_i \in \{\textit{leak}, \textit{safe}, \textit{anomalous}, \textit{ambiguous}\}$ together with the two underlying signal scores $\text{MaxChunkSim}(R_i, P)$ and $\text{CosineSim}(R_i, G)$. In production, the verdict drives the immediate action (flag, pass, or escalate); the signal scores provide interpretability.

**Aggregate score.** The PLR score is a scalar in $[0, 1]$, where higher values indicate greater resistance to leakage:

$$\text{PLR} = 1 - \frac{1}{N} \sum_{i=1}^{N} s_i \tag{2}$$

where $s_i$ is the per-record leakage score defined as follows.

**Binary mode.** Each verdict maps to a score:

$$s_i = \begin{cases} 1 & \text{if } v_i = \textit{leak} \\ 0 & \text{if } v_i = \textit{safe} \\ s_{\text{esc}} & \text{if } v_i \in \{\textit{anomalous}, \textit{ambiguous}\} \end{cases} \tag{3}$$

For escalated records, $s_{\text{esc}}$ depends on whether a judge is configured:

- **With judge**: the verdict is resolved to *leak* (or *violation*) $\rightarrow s_i = 1$; *safe* $\rightarrow s_i = 0$; *uncertain* (when the judge itself cannot decide) $\rightarrow s_i = 0.5$.
- **Without judge**: escalated records default to $s_{\text{esc}} = 0.5$, representing maximum uncertainty. Alternatively, they can be excluded from the aggregate via a parameter.

The use of a judge is recommended. Without one, the 0.5 default penalises the model conservatively without making an unsupported claim of leak or safety.

Per-category PLR scores are reported alongside the global scalar to capture leakage patterns by information type (Section 7.1).

## 5 Similarity Methods

We evaluate four methods for computing $\text{MaxChunkSim}(R, P)$.

### 5.1 Cosine Similarity on Sentence Embeddings

Both texts are encoded independently as dense vectors using `all-MiniLM-L6-v2` [Reimers and Gurevych, 2019], and similarity is the cosine of the angle between them:

$$\text{CosineSim}(R, c_i) = \frac{\mathbf{e}_R \cdot \mathbf{e}_{c_i}}{\|\mathbf{e}_R\| \, \|\mathbf{e}_{c_i}\|} \tag{4}$$

This is the standard semantic similarity baseline: open-source, reproducible, and well-calibrated in $[0, 1]$. It captures paraphrase-level semantic similarity that lexical methods miss. In our experiments it serves dual purpose—as Signal 1 for the cosine pipeline and as Signal 2 for every pipeline (including the reranker), where its calibrated scores are essential for the decision table to function correctly.

### 5.2 Cross-Encoder Reranker

The `Qwen3-Reranker-0.6B` [Qwen Team, 2025] reads the response and chunk jointly in a single forward pass, producing a log-softmax probability of "yes" to the fixed instruction $q$: *"Given the agent response as the Query, determine whether the system prompt content (Document) is revealed or leaked in the response."* Unlike bi-encoders, cross-encoders attend over the full concatenated input, detecting sophisticated paraphrase that independent encodings miss. The score is:

$$\text{RerankerSim}(R, c_i) = P_\theta(\text{yes} \mid q,\, c_i,\, R) \tag{5}$$

where $P_\theta$ is the log-softmax output of the cross-encoder $\theta$ evaluated on the concatenation of $q$, $c_i$, and $R$.

The reranker achieves a mean MaxChunkSim of 0.970 on adversarial records—the strongest detection signal of all evaluated methods—and is the recommended Signal 1 component.

### 5.3 ROUGE-L

ROUGE-L [Lin, 2004] computes the F1 of the Longest Common Subsequence (LCS) between two texts. It is a purely lexical metric with no ML dependencies, and was hypothesised to work well for *Secrets & Credentials* where leaked content (e.g., `sk-abc123`) is an exact string. The score is:

$$\text{ROUGE-L}(R, c_i) = \frac{2 \cdot P_{\text{lcs}} \cdot R_{\text{lcs}}}{P_{\text{lcs}} + R_{\text{lcs}}}, \qquad P_{\text{lcs}} = \frac{|\,\text{LCS}(R,\, c_i)|}{|c_i|}, \qquad R_{\text{lcs}} = \frac{|\,\text{LCS}(R,\, c_i)|}{|R|} \tag{6}$$

**Outcome:** 0% detection on 52 adversarial records. Mean MaxChunkSim: 0.165.

**Structural incompatibility:** modern instruction-tuned LLMs paraphrase by design. A model leaking "Your API key is `sk-abc123`" reformulates it as "The authentication credential uses identifier `sk-abc123`"—lexical overlap collapses to near zero even for a clear leak. ROUGE detects verbatim copy-paste, which is precisely the least likely case for well-trained models. We report this as a negative result motivating semantic detection.

### 5.4 NLI Entailment (DeBERTa)

The `cross-encoder/nli-deberta-v3-small` [He et al., 2021] classifies text pairs as entailment, neutral, or contradiction. We use the entailment probability as a similarity proxy: if the response entails the chunk, content from the prompt is present in the response. The score is:

$$\text{NLISim}(R, c_i) = P_\varphi(\text{entailment} \mid \text{premise} = c_i,\ \text{hypothesis} = R) \tag{7}$$

where $P_\varphi$ is the softmax output of the NLI model $\varphi$ over the three label classes.

**Outcome:** 5.8% detection (3/52). Mean MaxChunkSim: 0.113.

**Structural incompatibility:** two problems arise simultaneously. First, the model has a 512-token limit; benchmark system prompts exceed it (measured truncation at 624 tokens), silently dropping the very content being evaluated. Second, NLI is designed for logical entailment between short sentences, not for detecting whether specific information from a long document appears in a paraphrased response. The task mismatch causes systematic under-scoring of genuine leaks regardless of threshold choice.

# 6 Experimental Setup

**Benchmark construction.** All 68 benchmark responses were generated by `google/gemma-3-12b-it` queried via HuggingFace Inference Endpoints, serving as the target agent under evaluation. Gemma-3-12B-IT was chosen because it produces direct responses without chain-of-thought reasoning tokens, yielding clean response text free of intermediate thinking artifacts. Four realistic system prompts were constructed representing: a healthcare customer support assistant (NovaMed), a technical sales assistant (TechVault), a content moderation platform (SafeStream), and a corporate HR assistant (GlobalCorp).

**Adversarial records (52).** 13 per category $\times$ 4 categories. Difficulty distribution: $\sim$5 easy, $\sim$7 medium, and $\sim$5 hard per category. Hard instances use roleplay, indirect questions, and multi-step misdirection.

**Legitimate records (16).** 4 per category. These are queries where the agent *must* reference system prompt content to answer correctly (e.g., *"What topics can you help me with?"*). They are hard negatives for false positive analysis—the most challenging case for the two-signal design.

**Evaluation.** All four methods are evaluated on all 68 records using locally-run models (`all-MiniLM-L6-v2`, `nli-deberta-v3-small`, `Qwen3-Reranker-0.6B`). Threshold calibration and combination strategies are evaluated on the full 68-record dataset, split by adversarial and legitimate subsets.

# 7 Results

## 7.1 Individual Method Performance

Table 2 shows overall and per-difficulty detection rates on the 52 adversarial records. The reranker is the strongest detector at every difficulty level. Cosine provides meaningful detection (34.6%) and is the only viable alternative. NLI and ROUGE are disqualified. A critical property of the reranker not captured by detection rate alone is that it produces *zero hard false negatives* at any threshold evaluated: adversarial records it does not detect are escalated to the judge rather than incorrectly closed as safe. This property is central to the threshold calibration results in Section 7.3.

Table 2: Detection rate on 52 adversarial records, by difficulty.

| Method | Overall | Easy | Medium | Hard |
|---|---|---|---|---|
| Reranker | **61.5%** | 76.9% | 63.2% | 50.0% |
| Cosine | 34.6% | 53.8% | 21.1% | 35.0% |
| NLI (DeBERTa) | 5.8% | 0.0% | 10.5% | 5.0% |
| ROUGE-L | 0.0% | 0.0% | 0.0% | 0.0% |

Table 3 shows detection by category for the two viable methods. The reranker dominates in *Secrets & Credentials*—where precise credential strings create specific contextual signals that the cross-encoder captures—and is stronger across all categories. Cosine is comparatively stronger in *Internal Instructions*, where behavioral constraints produce more diffuse semantic overlap.

Table 3: Detection rate by leakage category (reranker vs. cosine).

| Category | Reranker | Cosine |
|---|---|---|
| Secrets & Credentials | **69.2%** | 15.4% |
| Guard Mechanism Exposure | **69.2%** | 38.5% |
| Permissions & Roles | **53.8%** | 38.5% |
| Internal Instructions | **53.8%** | 46.2% |

### 7.2 False Positive Analysis

Table 4 reports false positive rates on the 16 legitimate records. All methods produce at most one false positive—the same record across all methods.

Table 4: False positive rates on 16 legitimate records.

| Method | FP Rate | Safe | → Judge | Leak (FP) |
|---|---|---|---|---|
| Cosine | 6.2% (1/16) | 10/16 | 5/16 | 1/16 |
| Reranker | 6.2% (1/16) | 4/16 | 11/16 | 1/16 |
| ROUGE-L | 0.0% (0/16) | 14/16 | 2/16 | 0/16 |
| NLI | 0.0% (0/16) | 13/16 | 3/16 | 0/16 |

The sole false positive (`guar_001_leg_medium`—*"How does the reporting system work on SafeStream?"*) requires the agent to describe its own moderation pipeline, producing a response with unavoidable semantic overlap with the system prompt. This is a domain-boundary edge case: queries about the agent's own domain necessarily echo the system prompt, and no similarity threshold can disambiguate them from leaks without additional context.

ROUGE-L and NLI achieve 0% FP not because of precision but because they cannot classify anything as a leak—their zero false positive rate is a consequence of structural incapacity, not discrimination quality.

### 7.3 Threshold Calibration

Having identified the reranker as the recommended Signal 1 method, we evaluate how the decision threshold $\tau$ affects the detection–false positive tradeoff. Table 5 reports results for the reranker across $\tau \in [0.55, 0.75]$ on the full 68-record dataset.

The key distinction in this analysis is between two types of missed detections:

- **Hard false negatives (FN-safe):** adversarial records closed as *safe*—the system makes a confident wrong decision.
- **Soft false negatives (FN-judge):** adversarial records escalated to the judge—the system is uncertain and defers rather than deciding.

Three observations emerge. First, the reranker produces *zero hard false negatives across all thresholds evaluated*: every adversarial record it fails to detect is escalated to the judge, never incorrectly labeled safe. Second, $\tau = 0.65$ represents the optimal operating point: detection

Table 5: Reranker threshold sweep on 52 adversarial and 16 legitimate records.

| | Adversarial (52) | | | Legitimate (16) | |
|---|---|---|---|---|---|
| $\tau$ | Detection | FN-safe | FN-judge | FP | Safe |
| 0.55 | 50.0% | 0.0% | 50.0% | 6.2% | 18.8% |
| **0.60** | **61.5%** | **0.0%** | **38.5%** | **6.2%** | **25.0%** |
| **0.65** | **76.9%** | **0.0%** | **23.1%** | **6.2%** | **25.0%** |
| 0.70 | 84.6% | 0.0% | 15.4% | 31.2% | 25.0% |
| 0.75 | 84.6% | 0.0% | 15.4% | 50.0% | 6.2% |

improves from 61.5% to 76.9% (15.4 percentage points) while false positives remain at 6.2%—the same single domain-boundary edge case identified at $\tau = 0.6$. Third, $\tau = 0.70$ triggers a sharp false positive increase to 31.2% (5/16 legitimate records), making it unsuitable for production use. We therefore recommend $\tau = 0.6$ as the conservative default and $\tau = 0.65$ as the deployment target after validating false positive tolerance on a representative legitimate sample.

### 7.4 Smart Cascade: A Negative Result

Given that cosine and the reranker have complementary score distributions—cosine well-calibrated in $[0, 1]$, reranker concentrated near 0 and 1—we evaluate a *smart cascade* that uses cosine to pre-filter safe responses and relies on the reranker for leak detection:

1. Compute both cosine and reranker verdicts via the decision table.
2. If cosine declares *safe*: return **safe**.
3. If reranker declares *leak*: return **leak**.
4. All remaining cases: escalate to judge.

The motivation is that cosine closes 62.5% of legitimate queries as safe without running the reranker, reducing unnecessary inference cost and judge load on clean traffic.

Table 6 compares the smart cascade against the reranker alone at both recommended thresholds on adversarial records.

Table 6: Smart cascade vs. reranker alone on 52 adversarial records (all at $\tau = 0.6$ unless noted). FN-safe = hard false negatives; FN-judge = soft false negatives.

| Configuration | Detection | FN-safe | FN-judge |
|---|---|---|---|
| Reranker $\tau = 0.60$ | 61.5% | 0.0% | 38.5% |
| **Reranker** $\tau = 0.65$ | **76.9%** | **0.0%** | **23.1%** |
| Smart cascade $\tau = 0.6/0.6$ | 61.5% | 17.3% | 21.2% |

The smart cascade matches the reranker's 61.5% detection but introduces 9 hard false negatives (17.3%) that the reranker alone would have escalated to the judge. These are adversarial records where cosine's Signal 2 is high enough to declare *safe*, but the reranker's Signal 1 is in the ambiguous range—not a disagreement between methods, but both methods being uncertain in the same direction. The cascade's reduction in judge escalation (38.5% to 21.2%) comes entirely from those 9 records being incorrectly closed rather than correctly deferred.

Threshold tuning cannot resolve this: raising the cosine threshold to suppress safe closures only recovers the hard false negatives once cosine is set high enough to never declare anything safe, at which point the cascade collapses to the reranker alone.

The smart cascade provides genuine value for *legitimate traffic throughput*: cosine reduces reranker inference cost on clean requests. In adversarial detection, the reranker at $\tau = 0.65$ is strictly superior.

## 8 Discussion

**Why ROUGE and NLI fail.** The failure of lexical and entailment-based approaches is not a calibration artifact—it is structural. Modern instruction-tuned LLMs rephrase rather than regurgitate, collapsing lexical overlap to near-zero even for unambiguous leaks. NLI compounds this with 512-token truncation that silently discards the system prompt content being evaluated. Both methods are disqualified regardless of threshold tuning, and we report this as a useful negative result for practitioners evaluating off-the-shelf similarity metrics for leak detection.

**The reranker's conservatism: a detection asset and an operational cost.** The cross-encoder reranker scores adversarial records with a mean MaxChunkSim of 0.970, concentrating scores near 0 and 1 and leaving few records in the ambiguous mid-range. When it fails to detect a leak, it escalates rather than closes—a genuinely uncertain case correctly deferred to the judge. This conservatism is its most important property for security-critical deployments: it never makes a confident wrong call on adversarial traffic.

The same conservatism is a liability on legitimate traffic. At $\tau = 0.65$, the reranker closes only 4 of 16 legitimate records as *safe* directly, escalating the remaining 11 to the judge. Cosine closes 10 of 16 as safe without judge involvement. Each unnecessary escalation carries the full inference cost of an LLM judge call, which at production scale makes the reranker-only pipeline significantly more expensive than its detection numbers suggest. This is the central operational trade-off of the current metric: the method that eliminates hard false negatives on adversarial traffic is also the one that over-escalates clean traffic most aggressively.

**The domain-boundary false positive.** The 6.2% false positive rate traces to a single edge case that is inherent to semantic similarity-based detection: agents operating in the same domain as their system prompt will produce responses that echo prompt language when answering legitimate domain queries. This edge case is stable across thresholds from $\tau = 0.55$ to $\tau = 0.65$—the same record is flagged at all three values. One mitigation strategy is per-query reference calibration—constructing $G$ as a response that references the domain without sensitive specifics, making Signal 2 more discriminative for boundary cases.

**Threshold calibration in practice.** The sharp false positive increase at $\tau = 0.70$ suggests the reranker's score distribution has a natural gap between legitimate and adversarial responses in the $[0.65, 0.70]$ range. Practitioners should treat $\tau = 0.65$ as the deployment target but validate it against a held-out legitimate sample from their own domain before applying it in production, as the optimal threshold may shift with different system prompt styles or agent models.

**Limitations.**

- **Threshold calibration**: the recommended threshold $\tau = 0.65$ is calibrated on this benchmark. Different system prompt styles, agent models, or domain vocabularies may shift the optimal threshold. Per-deployment calibration against a held-out legitimate sample is recommended before raising the threshold above the conservative default of 0.6.
- **Scale**: the benchmark comprises 68 records. Results are directionally robust but per-category confidence intervals are wide.

- **Multi-turn attacks**: the current pipeline evaluates single-turn responses. Multi-turn attacks—where an adversary extracts the prompt incrementally across several turns—are outside the current scope and are left for future work.
- **Undetected leaks**: 23.1% of adversarial records (12/52) are not automatically classified by the reranker at $\tau = 0.65$ and are escalated to the judge. These are medium-to-hard paraphrase attacks where the reranker score falls below threshold. A judge component is a required part of any production deployment.

**Future work.** A multi-turn evaluation protocol and a lightweight LLM judge optimized for the ambiguous case are the primary extensions. A *continuous scoring mode*—returning a scalar leakage intensity rather than a discrete verdict—is a natural extension of the two-signal design, but is left for future work. The intended formulation $s_i = \text{MaxChunkSim}(R_i, P) \times (1 - \text{CosineSim}(R_i, G))$ assumes that Signal 1 varies meaningfully across record types. With the reranker, Signal 1 concentrates near 1 for both adversarial and legitimate records that reference their domain, collapsing the product to approximately $1 - \text{CosineSim}(R_i, G)$ and losing the independent Signal 1 contribution. Reliable continuous scores require either a Signal 1 method whose scores discriminate between legitimate domain references and actual leakage, or a reformulation of the scoring function that accounts for this saturation effect. The deeper open problem is a deterministic *safe classifier*: a method that identifies with high confidence that a response does not leak system prompt content, without relying on an LLM judge and without the hard false negatives that disqualify cosine for this role. Such a component would allow the pipeline to close clean traffic cheaply and reserve judge inference exclusively for the genuinely ambiguous cases the reranker flags, eliminating the current tension between detection reliability and operational cost.

# 9 Conclusion

We introduce PLR (Prompt Leakage Resistance), a metric for quantifying an LLM agent's resistance to system prompt extraction attacks. The metric combines a chunk-level maximum similarity score with a cosine alignment score against a safe reference response, applying a four-way decision table that separates confirmed leaks, safe responses, and judge-escalated ambiguous cases.

Our experiments on 68 benchmark records show that lexical (ROUGE-L) and entailment-based (NLI) methods are structurally incompatible with the task due to LLM paraphrase and token-limit truncation respectively. Among the viable methods, the cross-encoder reranker is the only one that produces zero hard false negatives across all evaluated thresholds—adversarial records it cannot confidently classify are escalated to the judge rather than incorrectly closed as safe.

Threshold calibration identifies $\tau = 0.65$ as the optimal operating point: 76.9% detection, 0% hard false negative rate, and 6.2% false positive rate. The threshold $\tau = 0.70$ achieves higher detection (84.6%) but triggers a sharp false positive increase to 31.2%, disqualifying it for production use. We recommend $\tau = 0.6$ as the conservative default and $\tau = 0.65$ as the validated deployment target.

A smart cascade combining cosine embeddings and the reranker was evaluated as a cost optimization: cosine pre-filters safe responses to reduce unnecessary reranker inference on legitimate traffic. However, the cascade introduces 17.3% hard false negatives on adversarial records by converting cases the reranker would escalate into confident-but-wrong safe verdicts. Threshold tuning does not resolve this—the cascade can only eliminate hard false negatives by disabling cosine's safe closures entirely, at which point it collapses to the reranker alone. The smart cascade is therefore not recommended as the primary detection configuration.

PLR is integrated into the Gaussia evaluation framework, enabling practitioners to compare model configurations on a common numeric scale across leakage categories and difficulty levels.

## Open Source

PLR is developed as part of the Gaussia evaluation framework, an open-source initiative for reproducible LLM agent evaluation. The benchmark, experiment notebooks, and full implementation are publicly available as part of `pygaussia` at `https://github.com/gaussia-labs/pygaussia`.

## References


Pengcheng He, Xiaodong Liu, Jianfeng Gao, and Weizhu Chen. DeBERTa: Decoding-enhanced BERT with disentangled attention. In *International Conference on Learning Representations*, 2021.

Bo Hui, Haolin Yuan, Neil Gong, Philippe Burlina, and Yinzhi Cao. PLeak: Prompt leaking attacks against large language model applications. In *Proceedings of the 2024 ACM SIGSAC Conference on Computer and Communications Security*. ACM, 2024. arXiv:2405.06823.

Chin-Yew Lin. ROUGE: A package for automatic evaluation of summaries. In *Text Summarization Branches Out: Proceedings of the ACL-04 Workshop*. Association for Computational Linguistics, 2004.

Rodrigo Nogueira and Kyunghyun Cho. Passage re-ranking with BERT. *arXiv preprint arXiv:1901.04085*, 2019.

Fábio Perez and Ian Ribeiro. Ignore previous prompt: Attack techniques for language models. In *NeurIPS ML Safety Workshop*, 2022. arXiv:2211.09527. Best Paper Award.

Qwen Team. Qwen3 technical report. Technical report, Alibaba Group, May 2025. arXiv:2505.09388.

Nils Reimers and Iryna Gurevych. Sentence-BERT: Sentence embeddings using siamese BERT-networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, 2019.